

Esse código é uma ferramenta automatizada que analisa a viabilidade financeira de investir em energia solar. Ele "lê" a sua conta de luz em PDF para descobrir o seu consumo e tarifa, cruza esses dados com a média de horas de sol da sua região e calcula o tamanho ideal do sistema fotovoltaico necessário. Por fim, o script projeta os custos de instalação e a economia mensal estimada, entregando um relatório completo com gráficos que mostram em quanto tempo o investimento inicial se pagará.

Esse código ainda está em desenvolvimento e não está lendo corretamente as contas de energia, o código feito separadamente para isso está calculando valores imprecisos e precisa de ajustes.

CÓDIGO

```
# -*- coding: utf-8 -*-  
"""código em desenvolvimento
```

Automatically generated by Colab.

Original file is located at

```
https://colab.research.google.com/drive/1eP86hXTPrWNaXIXee4Zi\_NagASqVtXEh  
"""
```

```
import matplotlib.pyplot as plt  
import matplotlib.patches as mpatches  
import matplotlib.gridspec as gridspec  
import numpy as np  
import os  
import pdfplumber  
import pytesseract  
from PIL import Image  
import re  
from io import BytesIO
```

```
# Tenta importar o uploader do Colab; se não estiver disponível, define  
fallback
```

```
try:  
    from google.colab import files as colab_files  
    COLAB = True  
except ImportError:  
    COLAB = False
```

```
# _____  
# PALETA DE CORES  
# _____
```

```
COR_SOL      = "#FFB300"
COR_VERDE    = "#2ECC71"
COR_AZUL     = "#1A73E8"
COR_CINZA    = "#BDC3C7"
COR_FUNDO    = "#0D1117"
COR_PAINEL   = "#161B22"
COR_TEXTO    = "#E6EDF3"
COR_ACENTO   = "#F97316"
COR_VERMELHO = "#E74C3C"
```

```
# -----
# TABELA DE IRRADIAÇÃO SOLAR (horas de sol pleno/dia - HSP)
# Fonte: CRESESB / Atlas Brasileiro de Energia Solar (INPE)
# -----
# Chaves em minúsculas para facilitar busca
IRRADIACAO_CIDADES = {
    # Rio Grande do Sul
    "alegrete": 4.8,
    "bagé": 4.9,
    "caxias do sul": 4.6,
    "cruz alta": 4.8,
    "erechim": 4.5,
    "gramado": 4.5,
    "ijuí": 4.8,
    "novo hamburgo": 4.6,
    "passo fundo": 4.7,
    "pelotas": 4.7,
    "porto alegre": 4.6,
    "rio grande": 4.6,
    "santa cruz do sul": 4.7,
    "santa maria": 4.8,
    "santana do livramento": 5.0,
    "são leopoldo": 4.6,
    "três passos": 4.7,
    "uruguaiana": 5.0,
    # Santa Catarina
    "blumenau": 4.5,
    "chapecó": 4.6,
    "criciúma": 4.4,
    "florianópolis": 4.5,
    "itajaí": 4.5,
    "joinville": 4.4,
    "lages": 4.6,
    "tubarão": 4.4,
```

```
# Paraná
"curitiba":      4.5,
"cascavel":      4.9,
"foz do iguaçu": 5.1,
"londrina":      5.1,
"maringá":       5.2,
"ponta grossa":  4.7,
# São Paulo
"campinas":      5.0,
"ribeirão preto": 5.3,
"santos":        4.8,
"são paulo":     4.7,
"sorocaba":      5.0,
# Rio de Janeiro
"campos dos goytacazes": 5.2,
"niterói":       5.0,
"rio de janeiro": 5.0,
"volta redonda": 5.1,
# Minas Gerais
"belo horizonte": 5.3,
"juiz de fora":  5.0,
"montes claros": 5.8,
"uberlândia":    5.5,
"uberaba":       5.6,
# Bahia
"feira de santana": 5.5,
"ilhéus":         5.0,
"salvador":       5.2,
"vitória da conquista": 5.5,
# Outros estados
"belém":         4.9,
"belo horizonte": 5.3,
"brasília":      5.7,
"cuiabá":        5.6,
"fortaleza":     5.7,
"goiânia":       5.6,
"macapá":        5.0,
"maceió":        5.6,
"manaus":        4.8,
"natal":         6.0,
"palmas":        5.6,
"porto velho":   4.8,
"recife":        5.7,
"rio branco":    4.7,
"são luís":      5.6,
```

```

        "teresina":          5.9,
        "vitória":          5.0,
    }

# Mapeamento de UF → HSP médio estadual (fallback se cidade não for
encontrada)
IRRADIACAO_UF = {
    "AC": 4.7, "AL": 5.6, "AM": 4.8, "AP": 5.0, "BA": 5.4,
    "CE": 5.7, "DF": 5.7, "ES": 5.0, "GO": 5.6, "MA": 5.5,
    "MG": 5.3, "MS": 5.4, "MT": 5.6, "PA": 4.9, "PB": 5.8,
    "PE": 5.7, "PI": 5.9, "PR": 4.9, "RJ": 5.0, "RN": 6.0,
    "RO": 4.8, "RR": 4.9, "RS": 4.7, "SC": 4.5, "SE": 5.5,
    "SP": 4.9, "TO": 5.6,
}

def buscar_hsp(cidade: str | None, uf: str | None) -> tuple[float | None,
str]:
    """
    Retorna (hsp, fonte_descricao).
    Tenta cidade primeiro, depois UF, depois retorna None.
    """
    if cidade:
        cidade_norm = cidade.strip().lower()
        # Busca exata
        if cidade_norm in IRRADIACAO_CIDADES:
            hsp = IRRADIACAO_CIDADES[cidade_norm]
            return hsp, f"tabela CRESESB - {cidade.title()}"
        # Busca parcial (ex: "Porto Alegre/RS" → "porto alegre")
        for chave, hsp in IRRADIACAO_CIDADES.items():
            if chave in cidade_norm or cidade_norm in chave:
                return hsp, f"tabela CRESESB - {chave.title()}"
        (aproximado)"

    if uf:
        uf_norm = uf.strip().upper()
        if uf_norm in IRRADIACAO_UF:
            hsp = IRRADIACAO_UF[uf_norm]
            return hsp, f"média estadual {uf_norm} (CRESESB)"

    return None, "não encontrada"

# _____
# EXTRAÇÃO DO PDF

```

```

# -----

def _extrair_texto_pdf(pdf_path: str) -> str:
    """Extrai texto do PDF com pdfplumber; fallback para OCR."""
    text = ""
    try:
        with pdfplumber.open(pdf_path) as pdf:
            for page in pdf.pages:
                t = page.extract_text(x_tolerance=2, y_tolerance=2)
                if t:
                    text += t + "\n"

        if not text.strip():
            print(" pdfplumber sem texto - tentando OCR com
pytesseract...")
            with pdfplumber.open(pdf_path) as pdf:
                if pdf.pages:
                    img = pdf.pages[0].to_image(resolution=300).original
                    buf = BytesIO()
                    img.save(buf, format="PNG")
                    buf.seek(0)
                    text = pytesseract.image_to_string(Image.open(buf),
lang="por")

    except Exception as e:
        print(f" Erro ao processar PDF: {e}")

    return text

def _extrair_consumo(text: str) -> float | None:
    """Tenta extrair consumo mensal em kWh do texto."""
    padroes = [
        # Distribuidoras que usam "Quantidade" na tabela de itens
        r"Quantidade\s+([0-9]+[. ,]?[0-9]*)\s+kWh",
        # CEEE / RGE / CPFL / Energisa padrão

        r"CONSUMO\s+(?:TOTAL|ATIVO|MENSAL)?\s*[:\s-]?\s*([0-9]+[. ,]?[0-9]*)\s*kWh",

        r"Energia\s+(?:El[eé]trica\s+)?Ativa\s+Fornecida.*?([0-9]+[. ,]?[0-9]*)\s*k
Wh",

        r"TOTAL\s+KWH\s+([0-9]+[. ,]?[0-9]*)",
        r"Consumo\s+(?:Total|M[eê]s)\s*[:\s-]?\s*([0-9]+[. ,]?[0-9]*)",
        # Genérico - número seguido de kWh (captura o maior valor
plausível)

```

```

        r"([0-9]{2,5}[.,]?[0-9]*)\s*kWh",
    ]
    for i, p in enumerate(padroes):
        m = re.search(p, text, re.IGNORECASE)
        if m:
            val = float(m.group(1).replace(",", "", "."))
            if 10 <= val <= 100_000: # faixa plausível
                print(f"    ✓ Consumo (padrão {i+1}): {val:.2f} kWh")
                return val
    print("    ✗ Consumo não encontrado.")
    return None

def _extrair_tarifa(text: str) -> float | None:
    """Tenta extrair tarifa em R$/kWh do texto."""
    padroes = [

        (r"Tarifa\s+(?:de\s+)?(?:Energia\s+)?Aplicada\s*[:\s-]?s*R?\$?\s*([0-9]+[.,][0-9]+)", "Tarifa Aplicada"),
        (r"Pre[cç]o\s+Unit[aá]rio\s*[:\s-]?s*R?\$?\s*([0-9]+[.,][0-9]+)", "Preço Unitário"),
        (r"([0-9]+[.,][0-9]+)\s*R\$/kWh", "R$/kWh inline"),
        (r"Tarifa\s*[:\s-]?s*R?\$?\s*([0-9]+[.,][0-9]+)", "Tarifa genérica"),
        (r"VALOR\s+KWH\s*R?\$?\s*([0-9]+[.,][0-9]+)", "VALOR KWH"),
    ]

    for p, nome in padroes:
        m = re.search(p, text, re.IGNORECASE)
        if m:
            val = float(m.group(1).replace(",", "", "."))
            if 0.05 <= val <= 5.0: # faixa plausível para tarifa BR
                print(f"    ✓ Tarifa ({nome}): R$ {val:.4f}/kWh")
                return val

    # Tenta TUSD + TE separados
    m = re.search(r"TUSD\s+([0-9]+[.,][0-9]+).*?TE\s+([0-9]+[.,][0-9]+)",
text, re.IGNORECASE | re.DOTALL)
    if m:
        tUSD = float(m.group(1).replace(",", "", "."))
        te = float(m.group(2).replace(",", "", "."))
        val = tUSD + te
        if 0.05 <= val <= 5.0:
            print(f"    ✓ Tarifa (TUSD {tUSD} + TE {te}): R$ {val:.4f}/kWh")

```

```

        return val

    # Inferência: Total a pagar ÷ Consumo
    print(" → Tentando inferir tarifa via Total a Pagar ÷ Consumo...")
    m_total = re.search(

r"(?:Total\s+a\s+[Pp]agar|VALOR\s+TOTAL|Total\s+da\s+[Ff]atura)\s*[:\-\]?\s
*R?\$?\s*([0-9]+[.][0-9]+)",
        text, re.IGNORECASE
    )
    consumo = _extrair_consumo(text)
    if m_total and consumo and consumo > 0:
        total = float(m_total.group(1).replace(",", "."))
        inf = total / consumo
        if 0.1 <= inf <= 3.0:
            print(f" ✓ Tarifa inferida: R$ {inf:.4f}/kWh (Total R$
{total:.2f} ÷ {consumo:.0f} kWh)")
            return inf

    print(" ✗ Tarifa não encontrada.")
    return None

def _extrair_cidade_uf(text: str) -> tuple[str | None, str | None]:
    """
    Tenta extrair a cidade e a UF do endereço de fornecimento
    presente na conta de luz.
    """
    # Padrões comuns em contas de energia brasileiras
    padroes_endereco = [
        # "Município: Porto Alegre UF: RS"

r"Munic[ií]pio\s*[:\-\]?\s*([A-ZÀ-Úa-zà-ú\s]+?)\s+UF\s*[:\-\]?\s*([A-Z]{2})"
,
        # "Cidade: Porto Alegre - RS"

r"(?:Cidade|Localidade)\s*[:\-\]?\s*([A-ZÀ-Úa-zà-ú\s]+?)\s*[-\/]\s*([A-Z]{2
})\b",
        # Endereço estilo "Rua X, 123 - Porto Alegre/RS" ou "Porto Alegre
- RS"

r"\b([A-ZÀ-Ú][a-zà-ú]+(?:\s+[A-ZÀ-Úa-zà-ú]+){0,4})\s*[/\-\]\s*([A-Z]{2})\b
",
        # "Local de fornecimento ... Porto Alegre RS"

```

```

r"(?:fornecimento|entrega|instala[cç][aã]o).*?([A-ZÀ-Ú][a-zà-ú]+(?:\s+[A-ZÀ-Úa-zà-ú]+){0,3})\s+([A-Z]{2})\b",
]

```

```

UFS_VALIDAS = set(IRRADIACAO_UF.keys())

```

```

for p in padroes_endereco:
    matches = re.findall(p, text, re.IGNORECASE | re.DOTALL)
    for m in matches:
        cidade_raw, uf_raw = m
        uf = uf_raw.strip().upper()
        if uf in UFS_VALIDAS:
            cidade = cidade_raw.strip().title()
            print(f"    ✓ Localização: {cidade} / {uf}")
            return cidade, uf

```

```

print("    ✗ Cidade/UF não identificadas no PDF.")
return None, None

```

```

def analisar_pdf_conta_energia(pdf_path: str) -> dict:
    """

```

```

    Extrai do PDF da conta de luz:

```

- consumo_mensal_kwh
- tarifa_kwh
- cidade, uf
- horas_sol_dia (buscado via tabela de irradiação)

```

    Retorna dict com os valores encontrados (None para os não encontrados).
    """

```

```

    print("\n" + "-" * 55)
    print("    📄 Analisando PDF da conta de energia...")
    print("-" * 55)

```

```

    text = _extrair_texto_pdf(pdf_path)

```

```

    if not text.strip():
        print("    ✗ Não foi possível extrair texto do PDF.")
        return {}

```

```

    # Debug: primeiras linhas

```

```

    linhas = [l for l in text.splitlines() if l.strip()]
    print("\n  [ Primeiras linhas extraídas ]")
    for l in linhas[:15]:
        print("    ", l)

```



```

print(" ...\\n")

consumo    = _extrair_consumo(text)
tarifa     = _extrair_tarifa(text)
cidade, uf = _extrair_cidade_uf(text)
hsp, fonte = buscar_hsp(cidade, uf)

resultado = {
    "consumo_mensal_kwh": consumo,
    "tarifa_kwh":        tarifa,
    "cidade":            cidade,
    "uf":                uf,
    "horas_sol_dia":     hsp,
    "hsp_fonte":         fonte,
}

print("\\n  — Resumo da extração —————")
print(f"  Consumo          : {consumo:.2f} kWh" if consumo else "
Consumo          : ✗ não encontrado")
print(f"  Tarifa            : R$ {tarifa:.4f}/kWh" if tarifa else "
Tarifa           : ✗ não encontrada")
print(f"  Localização       : {cidade} / {uf}" if cidade or uf else "
Localização      : ✗ não encontrada")
print(f"  Horas sol/dia      : {hsp:.1f} h/dia ({fonte})" if hsp else "
Horas sol/dia    : ✗ - informar manualmente")
print("  —————\\n")

return resultado

# —————
# UPLOAD DE PDF
# —————

def upload_pdf() -> str | None:
    if COLAB:
        uploaded = colab_files.upload()
        if uploaded:
            fname = list(uploaded.keys())[0]
            print(f"  Arquivo '{fname}' carregado.")
            return fname
        print("  Nenhum arquivo selecionado.")
        return None
    else:
        # Fora do Colab: solicita caminho manualmente
        path = input("  Informe o caminho completo do PDF: ")

```

```

").strip().strip('')
    if os.path.isfile(path):
        return path
    print(f" Arquivo não encontrado: {path}")
    return None

# -----
# COLETA DE DADOS COMBINADA (PDF + manual)
# -----
def coletar_dados() -> tuple[float, float, float, int]:
    """
    Tenta extrair dados do PDF e pede manualmente apenas o que
    não foi possível obter automaticamente.
    Retorna (consumo_kwh, tarifa_kwh, horas_sol, anos_simulacao).
    """
    print("\n" + "=" * 60)
    print(" ☀ SISTEMA DE ANÁLISE SOLAR FOTOVOLTAICA ☀")
    print("=" * 60)

    consumo = None
    tarifa = None
    horas_sol = None

    escolha = input("\nDeseja carregar um PDF da conta de energia? (s/n):")
    escolha = escolha.strip().lower()
    if escolha == "s":
        pdf_path = upload_pdf()
        if pdf_path:
            dados_pdf = analisar_pdf_conta_energia(pdf_path)
            consumo = dados_pdf.get("consumo_mensal_kwh")
            tarifa = dados_pdf.get("tarifa_kwh")
            horas_sol = dados_pdf.get("horas_sol_dia")

            if horas_sol:
                fonte = dados_pdf.get("hsp_fonte", "tabela")
                print(f" ☀ Irradiação obtida automaticamente:
{horas_sol:.1f} h/dia ({fonte})")

# — Solicita apenas os campos ainda ausentes —
print("\n" + "-" * 55)
print(" 📝 Completar dados em falta (Enter = pular se OK)")
print("-" * 55)

if consumo is None:

```

```

        consumo = float(input("\n📊 Consumo médio mensal (kWh):
").replace(",", "", "."))
    else:
        resp = input(f"\n📊 Consumo extraído do PDF: {consumo:.2f} kWh
[Enter para confirmar ou novo valor]: ").strip()
        if resp:
            consumo = float(resp.replace(",", "", "."))

    if tarifa is None:
        tarifa = float(input("💰 Tarifa de energia (R$/kWh) [ex: 0.85]:
").replace(",", "", "."))
    else:
        resp = input(f"💰 Tarifa extraída do PDF: R$ {tarifa:.4f}/kWh
[Enter para confirmar ou novo valor]: ").strip()
        if resp:
            tarifa = float(resp.replace(",", "", "."))

    if horas_sol is None:
        horas_sol = float(input("☀️ Média de horas de sol pleno/dia [ex:
4.7 para RS]: ").replace(",", "", "."))
    else:
        resp = input(f"☀️ Irradiação automática: {horas_sol:.1f} h/dia
[Enter para confirmar ou novo valor]: ").strip()
        if resp:
            horas_sol = float(resp.replace(",", "", "."))

    anos_simulacao = int(input("📅 Anos de simulação [ex: 25]: "))

    return consumo, tarifa, horas_sol, anos_simulacao

# -----
# CÁLCULOS PRINCIPAIS
# -----
def calcular_sistema(consumo_mensal_kwh, tarifa_kwh, horas_sol_dia,
anos_simulacao):
    consumo_diario_kwh = consumo_mensal_kwh / 30
    eficiencia_sistema = 0.80
    potencia_kwp = consumo_diario_kwh / (horas_sol_dia *
eficiencia_sistema)

    potencia_painel_kw = 0.450
    num_paineis = int(np.ceil(potencia_kwp / potencia_painel_kw))
    potencia_real_kwp = num_paineis * potencia_painel_kw

```

```

custo_por_kwp          = 4_500.00
custo_total            = potencia_real_kwp * custo_por_kwp

    geracao_mensal_kwh  = potencia_real_kwp * horas_sol_dia * 30 *
eficiencia_sistema
    economia_mensal     = geracao_mensal_kwh * tarifa_kwh
    reajuste_anual      = 0.06

meses_total            = anos_simulacao * 12
acumulado_economias    = []
economias_mensais      = []
saldo_acumulado        = []
payback_mes            = None

tarifa_atual           = tarifa_kwh
acumulado               = 0.0

for mes in range(1, meses_total + 1):
    if mes % 12 == 1 and mes > 1:
        tarifa_atual *= (1 + reajuste_anual)
    economia_mes        = geracao_mensal_kwh * tarifa_atual
    acumulado           += economia_mes
    saldo               = acumulado - custo_total
    economias_mensais.append(economia_mes)
    acumulado_economias.append(acumulado)
    saldo_acumulado.append(saldo)
    if saldo >= 0 and payback_mes is None:
        payback_mes = mes

payback_anos           = payback_mes / 12 if payback_mes else None
lucro_total             = acumulado_economias[-1] - custo_total
roi_pct                 = (lucro_total / custo_total) * 100

return {
    "consumo_mensal_kwh" : consumo_mensal_kwh,
    "tarifa_kwh"          : tarifa_kwh,
    "horas_sol_dia"       : horas_sol_dia,
    "potencia_kwp"        : potencia_kwp,
    "potencia_real_kwp"   : potencia_real_kwp,
    "num_paineis"         : num_paineis,
    "custo_total"         : custo_total,
    "geracao_mensal_kwh"  : geracao_mensal_kwh,
    "economia_mensal"     : economia_mensal,
    "payback_mes"         : payback_mes,
    "payback_anos"        : payback_anos,

```

```

        "acumulado_economias" : acumulado_economias,
        "economias_mensais"   : economias_mensais,
        "saldo_acumulado"     : saldo_acumulado,
        "lucro_total"         : lucro_total,
        "roi_pct"             : roi_pct,
        "anos_simulacao"      : anos_simulacao,
        "meses_total"         : meses_total,
        "reajuste_anual"      : reajuste_anual,
        "eficiencia_sistema"  : eficiencia_sistema,
    }

# -----
# RELATÓRIO NO TERMINAL
# -----
def exibir_relatorio(r):
    print("\n" + "=" * 60)
    print(" 📋 RELATÓRIO DE VIABILIDADE")
    print("=" * 60)
    print(f"\n 🛠️ SISTEMA DIMENSIONADO")
    print(f"      Potência necessária   : {r['potencia_kwp']:.2f} kWp")
    print(f"      Potência instalada     : {r['potencia_real_kwp']:.2f} kWp")
    print(f"      N° de painéis (450W)  : {r['num_paineis']} unidades")
    print(f"\n 🏠 FINANCEIRO")
    print(f"      Investimento total     : R$ {r['custo_total']:, .2f}")
    print(f"      Geração mensal        : {r['geracao_mensal_kwh']:.1f} kWh")
    print(f"      Economia 1° mês       : R$ {r['economia_mensal']:.2f}")
    print(f"      Reajuste tarifário    : {r['reajuste_anual']*100:.0f}% ao ano (projetado)")
    print(f"\n ⌚ RETORNO DO INVESTIMENTO")
    if r['payback_anos']:
        print(f"      Payback                : {r['payback_mes']} meses ({r['payback_anos']:.1f} anos)")
    else:
        print(f"      Payback                : não atingido no período simulado")
    print(f"\n 🏆 RESULTADO EM {r['anos_simulacao']} ANOS")
    print(f"      Total economizado      : R$ {r['acumulado_economias'][-1]:, .2f}")
    print(f"      Lucro líquido          : R$ {r['lucro_total']:, .2f}")
    print(f"      ROI                    : {r['roi_pct']:.1f}%")
    print(f"      CO₂ evitado (~)       : {r['geracao_mensal_kwh'] * r['meses_total'] * 0.0817:.0f} kg")
    if r['payback_anos'] and r['payback_anos'] <= r['anos_simulacao']:

```

```

        print(f"\n 🟢 VIÁVEL - o investimento se paga em
{r['payback_anos']:.1f} anos!")
    else:
        print(f"\n ⚠️ ATENÇÃO - payback não atingido no período
analísado.")
        print("=" * 60 + "\n")

# -----
# GRÁFICOS (mantidos do original)
# -----

def gerar_graficos(r):
    plt.rcParams.update({
        "font.family" : "DejaVu Sans",
        "axes.facecolor" : COR_PAINEL,
        "figure.facecolor" : COR_FUNDO,
        "text.color" : COR_TEXTO,
        "axes.labelcolor" : COR_TEXTO,
        "xtick.color" : COR_TEXTO,
        "ytick.color" : COR_TEXTO,
        "axes.edgecolor" : "#30363D",
        "grid.color" : "#21262D",
        "grid.linestyle" : "--",
        "grid.alpha" : 0.6,
    })

    meses = np.arange(1, r["meses_total"] + 1)
    anos_eixo = meses / 12

    fig = plt.figure(figsize=(18, 14))
    fig.suptitle("☀️ Análise de Viabilidade - Sistema Solar
Fotovoltaico",
                fontsize=20, fontweight="bold", color=COR_SOL, y=0.98)

    gs = gridspec.GridSpec(3, 3, figure=fig, hspace=0.45, wspace=0.38)

    # — Gráfico 1: Saldo acumulado (payback) —————
    ax1 = fig.add_subplot(gs[0, :2])
    ax1.set_title("Saldo Acumulado × Investimento (R$)", color=COR_TEXTO,
    fontsize=13, pad=10)
    saldo = np.array(r["saldo_acumulado"])
    positivo = np.where(saldo >= 0, saldo, np.nan)
    negativo = np.where(saldo < 0, saldo, np.nan)
    ax1.fill_between(anos_eixo, negativo, 0, color=COR_VERMELHO,
    alpha=0.25, label="Prejuízo")

```

```

    ax1.fill_between(anos_eixo, positivo, 0, color=COR_VERDE,
alpha=0.25, label="Lucro")
    ax1.plot(anos_eixo, saldo, color=COR_VERDE, linewidth=2.2)
    ax1.axhline(0, color=COR_CINZA, linewidth=1.2, linestyle="--")
    if r["payback_anos"]:
        ax1.axvline(r["payback_anos"], color=COR_SOL, linewidth=2,
linestyle=":",
                    label=f"Payback: {r['payback_anos']:.1f} anos")
    ax1.annotate(f"    Payback\n    {r['payback_anos']:.1f} anos",
                xy=(r["payback_anos"], 0),
                xytext=(r["payback_anos"] + 0.5, r["lucro_total"] *
0.15),
                color=COR_SOL, fontsize=10, fontweight="bold",
                arrowprops=dict(arrowstyle="->", color=COR_SOL,
lw=1.4))
    ax1.yaxis.set_major_formatter(
        plt.FuncFormatter(lambda x, _: f"R$ {x/1000:.0f}k" if abs(x) >=
1000 else f"R$ {x:.0f}"))
    ax1.set_xlabel("Anos"); ax1.set_ylabel("Saldo (R$)")
    ax1.legend(facecolor=COR_FUNDO, edgecolor="#30363D",
labelcolor=COR_TEXTO, fontsize=9)
    ax1.grid(True)

# — Gráfico 2: KPI Card —————
ax2 = fig.add_subplot(gs[0, 2])
ax2.set_facecolor(COR_PAINEL); ax2.axis("off")
kpis = [
    ("Investimento",    f"R$ {r['custo_total']:.0f}",
COR_ACENTO),
    ("Payback",        f"{r['payback_anos']:.1f} anos" if
r['payback_anos'] else "N/A", COR_SOL),
    ("ROI total",      f"{r['roi_pct']:.0f}%",
COR_VERDE),
    ("N° painéis",     str(r["num_paineis"]), COR_AZUL),
    ("Potência inst.", f"{r['potencia_real_kwp']:.2f} kWp",
COR_CINZA),
]
ax2.set_xlim(0, 1); ax2.set_ylim(0, 1)
for i, (label, valor, cor) in enumerate(kpis):
    y = 0.88 - i * 0.18
    ax2.text(0.05, y, label, fontsize=9, color=COR_CINZA,
va="center")
    ax2.text(0.05, y - 0.07, valor, fontsize=14, color=cor,
va="center", fontweight="bold")
ax2.set_title("Indicadores-chave", color=COR_TEXTO, fontsize=11,

```

```

pad=8)

# — Gráfico 3: Economia mensal ao longo do tempo —————
ax3 = fig.add_subplot(gs[1, :2])
ax3.set_title("Economia Mensal ao Longo do Tempo (R$)",
color=COR_TEXTO, fontsize=13, pad=10)
ax3.bar(anos_eixo, r["economias_mensais"], width=0.07,
        color=COR_AZUL, alpha=0.7, label="Economia / mês")
ax3.plot(anos_eixo, r["economias_mensais"], color=COR_SOL,
        linewidth=1.5, alpha=0.9, label="Tendência (reajuste anual)")
ax3.yaxis.set_major_formatter(plt.FuncFormatter(lambda x, _: f"R$
{x:.0f}"))
ax3.set_xlabel("Anos"); ax3.set_ylabel("Economia (R$)")
ax3.legend(facecolor=COR_FUNDO, edgecolor="#30363D",
labelcolor=COR_TEXTO, fontsize=9)
ax3.grid(True, axis="y")

# — Gráfico 4: Pizza - composição do custo —————
ax4 = fig.add_subplot(gs[1, 2])
ax4.set_title("Composição do Custo\ndo Sistema", color=COR_TEXTO,
fontsize=11, pad=8)
labels_pizza = ["Painéis\nFotovoltaicos", "Inversor", "Instalação\n&
Estrutura", "Cabeamento\n& Outros"]
sizes = [45, 20, 25, 10]
cores_p = [COR_SOL, COR_AZUL, COR_ACENTO, COR_CINZA]
explode = [0.04] * 4
wedges, texts, autotexts = ax4.pie(
    sizes, labels=labels_pizza, autopct="%1.0f%%",
    colors=cores_p, explode=explode,
    textprops={"color": COR_TEXTO, "fontsize": 8},
    wedgeprops={"edgecolor": COR_FUNDO, "linewidth": 1.5})
for at in autotexts:
    at.set_color(COR_FUNDO); at.set_fontweight("bold")

# — Gráfico 5: Geração vs Consumo - variação mensal —————
ax5 = fig.add_subplot(gs[2, :2])
ax5.set_title("Geração Solar × Consumo - Variação Mensal Estimada (Ano
1)",
        color=COR_TEXTO, fontsize=13, pad=10)
meses_nome =
["Jan", "Fev", "Mar", "Abr", "Mai", "Jun", "Jul", "Ago", "Set", "Out", "Nov", "Dez"]
fator_irrad =
np.array([1.15, 1.10, 1.05, 0.95, 0.85, 0.80, 0.82, 0.88, 0.95, 1.05, 1.10, 1.12])
geracao_var = r["geracao_mensal_kwh"] * fator_irrad
consumo_var = r["consumo_mensal_kwh"] * np.array(

```



```

[1.0,1.0,1.02,1.05,1.10,1.15,1.18,1.15,1.08,1.03,1.01,1.0])
x = np.arange(12); larg = 0.35
ax5.bar(x - larg/2, geracao_var, larg, label="Geração (kWh)",
color=COR_SOL, alpha=0.85)
ax5.bar(x + larg/2, consumo_var, larg, label="Consumo (kWh)",
color=COR_AZUL, alpha=0.85)
ax5.set_xticks(x); ax5.set_xticklabels(meses_nome, color=COR_TEXTO,
fontsize=9)
ax5.set_ylabel("kWh")
ax5.legend(facecolor=COR_FUNDO, edgecolor="#30363D",
labelcolor=COR_TEXTO, fontsize=9)
ax5.grid(True, axis="y")

# — Gráfico 6: Economia acumulada vs custo —————
ax6 = fig.add_subplot(gs[2, 2])
ax6.set_title("Acumulado\nvs Investimento", color=COR_TEXTO,
fontsize=11, pad=8)
anos_marco = [5, 10, 15, 20, 25]
economias_m = [r["acumulado_economias"][min(a*12-1,
r["meses_total"]-1)] for a in anos_marco]
cores_bar = [COR_VERDE if e >= r["custo_total"] else COR_VERMELHO
for e in economias_m]
bars = ax6.bar([str(a)+"a" for a in anos_marco], economias_m,
color=cores_bar, alpha=0.85, edgecolor=COR_FUNDO,
linewidth=1.2)
ax6.axhline(r["custo_total"], color=COR_SOL, linewidth=2,
linestyle="--",
label=f"Investimento\nR$ {r['custo_total']:.0f}")
for bar, val in zip(bars, economias_m):
ax6.text(bar.get_x() + bar.get_width() / 2,
bar.get_height() + r["custo_total"] * 0.02,
f"R${val/1000:.0f}k", ha="center", va="bottom",
color=COR_TEXTO, fontsize=8, fontweight="bold")
ax6.yaxis.set_major_formatter(plt.FuncFormatter(lambda x, _ :
f"R${x/1000:.0f}k"))
ax6.set_xlabel("Horizonte"); ax6.set_ylabel("R$")
ax6.legend(facecolor=COR_FUNDO, edgecolor="#30363D",
labelcolor=COR_TEXTO, fontsize=8)
ax6.grid(True, axis="y")

# — Rodapé —————
fig.text(0.5, 0.01,
"* Projeção estimada. Valores sujeitos a variações de
irradiação, tarifas e condições locais.",
ha="center", color=COR_CINZA, fontsize=8.5)

```

```

output_dir = "/mnt/user-data/outputs/"
os.makedirs(output_dir, exist_ok=True)
out_path = os.path.join(output_dir, "solar_viabilidade.png")
plt.savefig(out_path, dpi=150, bbox_inches="tight",
facecolor=COR_FUNDO)
print(f"  Gráficos salvos em: {out_path}")
plt.show()

```

```

# _____
# EXECUÇÃO PRINCIPAL
# _____

```

```

if __name__ == "__main__":
    consumo, tarifa, horas_sol, anos = coletar_dados()
    resultado = calcular_sistema(consumo, tarifa, horas_sol, anos)
    exhibir_relatorio(resultado)
    gerar_graficos(resultado)

```

```

# Instalar bibliotecas para leitura de PDF e OCR (reconhecimento de texto)
# pdfplumber é para PDFs com texto selecionável.
# pytesseract é para PDFs digitalizados (imagens de texto).
!pip install pdfplumber pytesseract Pillow
!sudo apt-get install tesseract-ocr

```

```

pip install PyPDF2

```

```

import re
import json
import PyPDF2

```

```

def extrair_texto_pdf(caminho_arquivo):
    """
    Abre um ficheiro PDF, lê todas as suas páginas e
    retorna o texto completo numa string.
    """
    texto_completo = ""
    try:
        with open(caminho_arquivo, 'rb') as arquivo_pdf:
            leitor = PyPDF2.PdfReader(arquivo_pdf)
            for pagina in leitor.pages:
                texto_pagina = pagina.extract_text()
                if texto_pagina:
                    texto_completo += texto_pagina + "\n"
    return texto_completo

```

```

except FileNotFoundError:
    return f"Erro: O ficheiro '{caminho_arquivo}' não foi encontrado."
except Exception as e:
    return f"Erro ao processar o PDF: {e}"

def analisar_fatura_completa(texto_fatura):
    """
    Analisa o texto completo de uma fatura de energia, extrai
    todas as informações relevantes e calcula a tarifa real.
    """
    if texto_fatura.startswith("Erro"):
        return {"erro": texto_fatura}

    dados = {
        "codigo_instalacao": None,
        "mes_referencia": None,
        "data_vencimento": None,
        "numero_medidor": None,
        "valor_total_documento": None,
        "diferenca_consumo_kwh": None,
        "custo_real_por_kwh": None
    }

    # 1. Extração do Código de Instalação
    match_instalacao = re.search(
        r'(?:(Código d[ae]\s+Instalação|CÓDIGO DE
INSTALAÇÃO) [\s\S]*?(\d{9,10}))',
        texto_fatura, re.IGNORECASE
    )
    if match_instalacao:
        dados["codigo_instalacao"] = match_instalacao.group(1)

    # 2. Extração do Mês de Referência
    match_mes = re.search(
        r'(?:(M[ÊË]S DE
REFER[ÊË]NCIA) [\s\S]*?([A-Za-z0-9\s/]+?) (?:\n|VENCIMENTO|"))',
        texto_fatura, re.IGNORECASE
    )
    if not match_mes:
        # Fallback para capturar mês/ano padronizado (ex: Maio / 2026,
        # Mai/26)
        match_mes = re.search(
            r'\b(?:Jan|Fev|Mar|Abr|Mai|Jun|Jul|Ago|Set|Out|Nov|Dez) [a-z]*\s*/\s*\d{2,4}
\b',

```

```

        texto_fatura, re.IGNORECASE
    )
    if match_mes:
        dados["mes_referencia"] = match_mes.group(0).strip().replace('"',
'').replace(' ', '')

    # 3. Extração da Data de Vencimento
    match_vencimento = re.search(
        r'(?:(Vencimento|DATA DE VENCIMENTO) [\s\S]*?(\d{2}/\d{2}/\d{4}))',
        texto_fatura, re.IGNORECASE
    )
    if match_vencimento:
        dados["data_vencimento"] = match_vencimento.group(1)

    # 4. Extração do Número do Medidor (Opcional, mas útil)
    match_medidor = re.search(
        r'(?:(Nº\s*do\s*Medidor|Nº\s*Medidor|Medidor) [\s\S]*?([A-Za-z0-9\-\+]))',
        texto_fatura, re.IGNORECASE
    )
    if match_medidor:
        # Limpa possíveis caracteres indesejados caso coleciono algo a
mais
        dados["numero_medidor"] = match_medidor.group(1).replace(':',
'').strip()

    # 5. Extração do Valor Total do Documento
    match_total = re.search(
        r'(?:(Total a pagar|TOTAL A PAGAR \(\RS\)|VALOR TOTAL(?: DO
DOCUMENTO| DA NOTA FISCAL)?)[^\d]*?(\d{1,5}(?:\.\d{3})*,\d{2}))',
        texto_fatura, re.IGNORECASE
    )
    if match_total:
        valor_limpo = match_total.group(1).replace('.', '').replace(',',
'.')
        dados["valor_total_documento"] = float(valor_limpo)

    # 6. Extração da Diferença de Consumo (kWh)
    match_consumo = re.search(
        r'(?:(Diferença de Leitura \(\Consumo\):|Consumo\s+faturado|Energia
Ativa Ativa-kWh|Energia Ativa TE[^\n]*?)\s*[\s\S]*?(\d+)\s*(?:kWh)?',
        texto_fatura, re.IGNORECASE
    )
    if not match_consumo:
        match_consumo = re.search(r'(\d+)\s*kWh', texto_fatura,

```

```

re.IGNORECASE)

    if match_consumo:
        dados["diferenca_consumo_kwh"] = int(match_consumo.group(1))

    #
    =====
    # 7. INTERAÇÃO SOLICITADA: Divisão do valor total pela diferença de
    consumo
    #
    =====

    if dados["valor_total_documento"] is not None and
    dados["diferenca_consumo_kwh"] is not None:
        if dados["diferenca_consumo_kwh"] > 0:
            calculo = dados["valor_total_documento"] /
            dados["diferenca_consumo_kwh"]
            dados["custo_real_por_kwh"] = round(calculo, 4)

    return dados

# =====
# TESTAR COM OS FICHEIROS PDF REAIS
# =====

# Insira aqui o nome do ficheiro que pretende processar.
# Pode testar com "fatura_rge_novo_modelo.pdf", "fatura_rge_modelo.pdf" ou
"conta-completa.pdf"
ficheiros_para_testar = [
    "fatura_rge_novo_modelo.pdf",
    "fatura_rge_modelo.pdf",
    "conta-completa.pdf"
]

for ficheiro in ficheiros_para_testar:
    print(f"\n--- PROCESSANDO: {ficheiro} ---")
    texto_extraido = extrair_texto_pdf(ficheiro)

    # Processa todas as informações se o ficheiro foi lido com sucesso
    if not texto_extraido.startswith("Erro"):
        resultado = analisar_fatura_completa(texto_extraido)
        print(json.dumps(resultado, indent=4, ensure_ascii=False))
    else:
        print(texto_extraido)

```